JAVA & OOP

# MCQ Revision Sheet

Collections | OOP Concepts | Coding Problem

21 Questions with Answers, Explanations & Full Code Solution

### How to use this sheet

Each question lists all options. The **correct option is shown in green with a checkmark**, followed by a short explanation in italics. Use the topic tag to focus revision on weak areas.

# SECTION A — MULTIPLE CHOICE QUESTIONS

## Q1 JAVA COLLECTIONS

**An application needs to store a collection of unique product codes that must always be automatically sorted in their natural (ascending) order. Which collection should be used?**

A. HashSet

✔ **B. TreeSet**

C. ArrayList

D. LinkedList

***Explanation:*** *TreeSet enforces uniqueness and keeps elements sorted in natural ascending order automatically (backed by a Red-Black tree).*

## Q2 CONSTRUCTORS & OVERLOADING

**A class "Employee" needs to support creating an object either with just a name, or with both a name and a salary. Which Java feature should the developer use to design the class this way?**

A. Method Overriding

✔ **B. Constructor Overloading, by defining multiple constructors with different parameter lists**

C. Static initialization blocks

D. Interface implementation

***Explanation:*** *Constructor overloading lets you define Employee(name) and Employee(name, salary) constructors in the same class.*

## Q3 HASHSET

**What is always true about a HashSet?**

A. Allows duplicate values

B. Maintains insertion order always

✔ **C. Stores unique elements**

D. Sorts elements automatically

***Explanation:*** *HashSet guarantees uniqueness only — it makes no guarantee about insertion order or sorted order.*

## Q4 HASHMAP BEHAVIOUR

**What will be printed? HashMap<Integer, String> map = new HashMap<>(); map.put(1, "A"); map.put(1, "B"); System.out.println(map.get(1));**

A. A

✔ **B. B**

C. AB

D. Null

***Explanation:*** *put(1, "B") overwrites the earlier put(1, "A") for the same key, so get(1) returns "B".*

## Q5 ARRAYLIST VS LINKEDLIST

**Which statement correctly describes a key difference between ArrayList and LinkedList in Java?**

A. ArrayList uses a doubly linked list internally, while LinkedList uses a dynamic array

B. LinkedList provides faster random access using an index compared to ArrayList

✔ **C. ArrayList provides efficient random access by index, while LinkedList can insert or remove elements at its ends without shifting other elements**

D. Both ArrayList and LinkedList have identical performance for all operations

***Explanation:*** *ArrayList (dynamic array) gives O(1) index access. LinkedList (doubly linked list) gives O(1) insert/remove at the ends without shifting.*

## Q6 METHOD OVERLOADING

**The application supports payment through UPI, Credit Card, and Net Banking. The developer wants multiple methods with the same name but different parameters. Which Java concept should be used?**

A. Method Overriding

B. Inheritance

C. Constructor

✔ **D. Method Overloading**

***Explanation:*** *Same method name, different parameter lists, within the same class — classic method overloading.*

## Q7 CHOOSING COLLECTIONS

**The platform stores Customer IDs and corresponding customer names. Requirements: search customer by ID, fast retrieval, one name per ID. Which collection is most suitable?**

✔ **A. HashMap**

B. HashSet

C. ArrayList

D. Queue

***Explanation:*** *A key-value mapping (ID → name) with O(1) average lookup is exactly what HashMap provides.*

## Q10 POLYMORPHISM

**A parent class "Shape" has a method area(). Subclasses "Circle" and "Rectangle" each override this method with their own implementation. At runtime, a Shape reference is used to call area() on different subclass objects, and the correct subclass version executes each time. Which OOP concept does this scenario best demonstrate?**

A. Compile time Polymorphism

✔ **B. Runtime Polymorphism (dynamic method dispatch)**

C. Static binding

D. Encapsulation

***Explanation:*** *The method call is resolved at runtime based on the actual object type — classic dynamic method dispatch via overriding.*

## Q11 LIST METHODS

**Which of the following method can be used to add the element in a List<>?**

✔ **A. add**

B. push

C. put

D. None of the above

***Explanation:*** *List<> uses add(element) to insert elements (push is for Deque/Stack, put is for Map).*

## Q12 METHOD OVERRIDING — OUTPUT

**What will be the output? class Parent{ void display(){ System.out.println("Parent"); } } class Child extends Parent{ void display(){ System.out.println("Child"); } } public class Main{ public static void main(String[] args){ Parent p = new Child(); p.display(); } }**

A. Parent

✔ **B. Child**

C. Compilation Error

D. Runtime Error

***Explanation:*** *p is declared as Parent, but the actual object is Child. Since display() is overridden, dynamic method dispatch calls Child's version at runtime.*

## Q13 OOP PRINCIPLES

**Which OOP principle is primarily responsible for achieving runtime flexibility by allowing a superclass reference to refer to a subclass object?**

A. Encapsulation

B. Abstraction

✔ **C. Polymorphism**

D. Inheritance

***Explanation:*** *This describes runtime (dynamic) polymorphism — a superclass reference pointing to a subclass object.*

## Q14 CHOOSING COLLECTIONS

**A university system needs to store student roll numbers as keys mapped to student objects, allowing very fast retrieval of a student's details using the roll number, and the order in which records are stored does not matter. Which collection is the most appropriate choice for this requirement?**

A. ArrayList, because it allows indexed access

✔ **B. HashMap, because it provides average constant time O(1) lookup using keys**

C. TreeSet, because it keeps elements sorted

D. LinkedList, because it allows fast insertion at both ends

***Explanation:*** *Key-based, order-independent, fast retrieval → HashMap with average O(1) lookup by key.*

## Q15 COLLECTION FRAMEWORK

**Which of the following will store only the unique keys in the Collection Framework?**

A. List

B. Set

✔ **C. Map**

D. None of the above

***Explanation:*** *A Map's keys are always unique by contract — each key maps to at most one value.*

## Q16 INTERFACES

**Which statement is correct about interfaces in Java?**

A. Objects cannot implement interfaces

✔ **B. Interfaces support abstraction**

C. Interfaces store instance variables only

D. Interfaces cannot have methods

***Explanation:*** *Interfaces define what a class must do without specifying how — the essence of abstraction.*

## Q17 INHERITANCE

**A child class extends a parent class containing method display(). Can the child object call it?**

A. No

B. Only if private

✔ **C. Yes**

D. Runtime Error

***Explanation:*** *A subclass inherits public/protected methods from its parent by default, so the child object can call display() directly.*

## Q18 CHOOSING COLLECTIONS

**Duplicate booking IDs must never be stored in the system. Which collection is most appropriate?**

A. ArrayList

B. LinkedList

✔ **C. HashSet**

D. Vector

***Explanation:*** *HashSet enforces uniqueness automatically, rejecting duplicate elements.*

## Q19 ENCAPSULATION DESIGN

**A team is designing classes for a Library Management System. They need to represent Books, where each book has a title, author and ISBN, and the internal fields should not be directly modified from outside the class but should be accessible through controlled methods. Which object oriented approach best fulfills this requirement?**

A. Make all fields public so any class can modify them directly

✔ **B. Apply encapsulation by declaring fields as private and providing public getter and setter methods**

C. Declare the class as static so fields are shared globally

D. Avoid using a class altogether and use only static methods

***Explanation:*** *Private fields + public getters/setters is the standard encapsulation pattern for controlled, indirect access.*

## Q20 PRIORITYQUEUE

**What happens when a PriorityQueue is iterated using an iterator?**

A. Elements are returned in sorted order

B. Elements are returned in insertion order

C. Elements are returned in priority order

✔ **D. No ordering guarantee is provided during iteration**

***Explanation:*** *PriorityQueue's iterator does NOT guarantee sorted/priority order — only poll()/remove() guarantees the highest-priority element. The internal heap array order is not the sorted order.*

# SECTION B — CODING PROBLEM

## Q21 JAVA CODING — COFFEE MANAGEMENT SYSTEM

You are tasked with designing the **Coffee Management System**. **Coffee class fields:**
id: integer  name: String  type: String  price: double

Make getter, setter and constructor for the Coffee class.

**Task 1:** Create a static method that returns the List of Coffee whose price is >= the average price of all coffee. Takes 1 input: List<Coffee> coffees. Print id, name, type and price of each coffee returned.

**Task 2:** Create a static method that returns the highest priced coffee based on a given type. Takes 2 inputs: List<Coffee> coffees and String type. Print id, name and price of the highest priced coffee in main. If no coffee is found with that type, print "No Coffee Found".

**Input format:** First line = number of coffees (n). Next n groups of 4 lines = id, name, type, price. Last line = type to search for Task 2.

### Sample Input 1

```
3
1
Davidoff
instant
1015.0
2
Barral-aged Coffee
ground
1000.0
3
Bru
instant
100.0
instant
```

### Sample Output 1

```
1
Davidoff
instant
1015.0
2
Barral-aged Coffee
ground
1000.0
1
Davidoff
1015.0
```

### Sample Input 2

```
0
```

### Sample Output 2

```
No Coffee Found
```

## Full Java Solution

```
import java.io.*;
import java.util.*;

class Coffee {
    private int id;
    private String name;
    private String type;
    private double price;

    public Coffee(int id, String name, String type, double price) {
        this.id = id;
        this.name = name;
        this.type = type;
        this.price = price;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getType() { return type; }
    public void setType(String type) { this.type = type; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}

public class Solution {

    // Task 1: coffees priced >= average price
    public static List<Coffee> getAboveAveragePriceCoffees(List<Coffee> coffees) {
        List<Coffee> result = new ArrayList<>();
        if (coffees == null || coffees.isEmpty()) return result;
        double total = 0.0;
        for (Coffee c : coffees) total += c.getPrice();
        double average = total / coffees.size();
        for (Coffee c : coffees) {
            if (c.getPrice() >= average) result.add(c);
        }
        return result;
    }

    // Task 2: highest priced coffee of a given type
    public static Coffee getHighestPricedCoffeeByType(List<Coffee> coffees, String type) {
        Coffee highest = null;
        if (coffees == null) return null;
        for (Coffee c : coffees) {
            if (c.getType() != null && c.getType().equalsIgnoreCase(type)) {
                if (highest == null || c.getPrice() > highest.getPrice()) {
                    highest = c;
                }
            }
        }
        return highest;
    }
```

```
    public static void main(String args[]) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine().trim());

        List<Coffee> coffees = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int id = Integer.parseInt(br.readLine().trim());
            String name = br.readLine().trim();
            String type = br.readLine().trim();
            double price = Double.parseDouble(br.readLine().trim());
            coffees.add(new Coffee(id, name, type, price));
        }

        String queryType = null;
        String line = br.readLine();
        if (line != null) queryType = line.trim();

        // Task 1
        List<Coffee> aboveAverage = getAboveAveragePriceCoffees(coffees);
        for (Coffee c : aboveAverage) {
            System.out.println(c.getId());
            System.out.println(c.getName());
            System.out.println(c.getType());
            System.out.println(c.getPrice());
        }

        // Task 2
        Coffee highest = getHighestPricedCoffeeByType(coffees, queryType);
        if (highest == null) {
            System.out.println("No Coffee Found");
        } else {
            System.out.println(highest.getId());
            System.out.println(highest.getName());
            System.out.println(highest.getPrice());
        }
    }
}
```